

ДЕКЛАРАЦИЯ ФОРМЫ

IMPERIUM NEW GENERATION

Owner-corrected candidate V0.2

Статус: канонический кандидат для регистрации в Doctrinarium. Не заменяет будущую машинную JSON/MD-зеркальную форму, но является главным человекочитаемым PDF-источником формы.

Назначение: удерживать цели, запреты, PASS/FAIL-критерии и принцип симбиоза IMPERIUM с агентами.

Главная формула

IMPERIUM должен перестать быть репозиторием правильных артефактов и стать кабиной управления силой: внешний хаос входит, органы и агенты фокусируют, Servitor исполняет, evidence доказывает, Custodes защищает, Throne впитывает только проверенное.

0. Статус документа и порядок применения

Этот документ является лучшей текущей формой Декларации Формы IMPERIUM New Generation. Он собран из целей Owner, черновика Servitor Prime, выводов Speculum и текущей линии NewGen. Его задача - не описать все навсегда, а задать устойчивую форму, по которой Servitor, Logos, Speculum и будущие органы должны понимать направление работы.

1. PDF-версия используется как надежный человекочитаемый источник, чтобы избежать mojibake и потери русских формулировок.
2. Для машинной работы позже должны быть созданы зеркала: JSON registry, краткий MD summary и Doctrinarium read-first pointer.
3. Пока Owner не сказал иначе, все будущие NewGen задачи должны сверяться с этой формой.
4. Финальная канонизация происходит только после регистрации через Doctrinarium и проверки Owner/Logos-Prime.

1. Главный тезис: что такое IMPERIUM New Generation

IMPERIUM_NEW_GENERATION - это чистая строительная зона будущего рабочего ядра IMPERIUM. Это не архив случайных файлов и не набор красивых названий органов. Это место, где строится суверенная машина фокусировки: она принимает хаос внешнего мира, задачи Owner, облачные и локальные ЛЛМ, инструменты, железо PC/ноутбука и агентные действия, а затем превращает все это в проверяемый результат с evidence, памятью, правилами, дифами, презентациями и непрерывным обучением системы.

Цель NewGen - получить clean core: рабочее ядро, которое можно атаковать, проверять, переносить, улучшать и в будущем поднимать до Throne/Core Agent на ноутбуке. Это ядро не должно загрязняться прямой работой агентов. Оно должно выдавать копии информации, принимать назад доказанные результаты, проверять их через Custodes/Inquisition/Doctrinarium и только затем впитывать approved truth.

2. Закон симбиоза IMPERIUM и агентов

Это постоянный закон линии, пока Owner явно не скажет иначе.

IMPERIUM функционирует в симбиозе с агентами. Сам IMPERIUM не является разговорной ЛЛМ. Он является машиной фокусировки, памяти, управления, доказательства, маршрутизации, защиты и обучения. Внешние облачные ЛЛМ, локальные Ollama/локальные ЛЛМ, CLI-агенты, Servitor, Logos, Speculum, Kiro, будущий Borya и другие сервисы являются агентами, которые работают в полном симбиозе с IMPERIUM.

Каждая задача, выполненная Servitor, обязана оставить в IMPERIUM осязаемую заботу и обучение. Недостаточно просто произвести локальный результат. Servitor должен, где возможно, регистрировать правила, скрипты, паттерны, lessons, pain gates, receipts, улучшения маршрутов, ограничения, evidence и подсказки для следующего агента.

1. Servitor исполняет задачу.
2. Servitor доказывает результат.
3. Servitor объясняет, что IMPERIUM узнал.
4. Servitor сохраняет reusable механизм или явно пишет, почему это невозможно.
5. Servitor не загрязняет ядро и не прячет мусор.
6. Следующая задача должна стать легче, точнее или безопаснее благодаря предыдущей.

3. Модель фокусирующей машины

IMPERIUM должен быть шлюзом и фильтром между внешним миром и результатом. Внешний мир включает облачные ЛЛМ, локальные ЛЛМ, интернет, OSS, базы данных, OSINT, deep research, инструменты, навыки, железо PC/ноутбука и любые операционно ценные источники. Эти источники не должны напрямую управлять ядром. Они проходят через фокусирующий слой IMPERIUM.

Вход	Роль IMPERIUM	Выход
Хаос внешнего мира, запрос Owner, ЛЛМ, интернет, OSS, базы, OSINT, железо	Фильтрация, декомпозиция, органы, директивы, выбор агента/инструмента, evidence gate, память боли	Цифровой продукт, код, приложение, игра, презентация, таблица, промпт, видео, отчет, доказанный результат

Фокусировка означает: выбрать правильную силу, дать ей правильный контекст, ограничить область работы, измерить цену, проверить результат, сохранить урок и не пустить грязь в чистое ядро.

4. Внутрянка и Внешка

Режим	Смысл	Выход
Internal / Внутрянка	Весь IMPERIUM работает на улучшение самого себя: Cockpit, органы, память, gates, hero hygiene, инструменты, маршруты, визуал, Doctrinarium.	Укрепление ядра, новые правила, scripts, pain memory, gates, архитектурная чистота.
External / Внешка	IMPERIUM выполняет freelance/client/digital задачи через копию или отдельный режим работы.	Сильнейшее доступное решение, evidence, презентация, diff history, pipeline story, плюсы/минусы, KPD-обоснование.

Лучшее решение не обязано быть идеальным. Оно обязано иметь максимальный KPD из доступных вариантов и честно показывать ограничения, минусы и цену.

5. Чистое ядро и будущий Throne

Clean core - это форма, где IMPERIUM уже способен проводить задачу через органы и агентов, получать директивы, видеть цену действий, контролировать железо, хранить историю, собирать continuity pack и не допускать fake green. После построения такого ядра оно должно пройти жесткий удар через Custodes/Speculum/Inquisition.

Будущий Throne/Core Agent на ноутбуке - это не простая копия репозитория. Это живой агент ядра, который управляет ноутбучным операционным контуром, повседневными задачами Owner и взаимодействием с другими агентами. Throne не принимает грязь напрямую. Он выдает clean information copy, получает результат, передает проверку Custodes, и только approved truth впитывается обратно.

6. Borya как будущий interface gateway

Borya - будущий Telegram/interface агент. Он не должен быть болталкой и не должен появиться раньше, чем есть Cockpit/action/event model. Его роль: принимать задачи, отдавать статусы, передавать вопросы Owner, запускать allowlisted действия, распределять нагрузку, отправлять continuity, приносить evidence и помогать управлять IMPERIUM дистанционно.

7. Обязательная роль Sanctum / Cockpit

Sanctum Operator Cockpit - это не декоративный dashboard. Это кабина управления IMPERIUM. Через нее Owner должен видеть систему, не лазая вручную по папкам, терминалам и reports.

Минимальная хорошая форма:

1. Owner открывает Sanctum одним URL.

2. Видит current task, кто работает, где blocker, next action.
3. Видит PC/VM2/VM3 routes и transfer truth.
4. Видит evidence, diff, changed files.
5. Может собрать continuity pack.
6. Может открыть development preview.
7. Видит вопросы Servitor/органов.
8. Может оставить Owner decision / stop / note.
9. Визуал не выглядит дефолтно.
10. Система не говорит green без evidence.
11. Боли становятся gates.
12. Органы реально влияют на работу.

8. Постоянная stable/candidate модель

У Owner должна быть постоянная стабильная точка запуска и отдельная кандидатная версия для разработки. Stable - это то, чем можно пользоваться. Candidate - это зона визуальных и функциональных экспериментов. Candidate заменяет stable только после доказательства: screenshots, validator, evidence, Owner acceptance, no regression, performance sanity.

Track	Назначение	Правило
Stable	Постоянная рабочая версия Cockpit.	Не ломается экспериментами. Обновляется только после promotion gate.
Candidate	Зона активной разработки визуала/функций.	Может меняться, но обязана иметь screenshots, diff, report, rollback/promotion rule.

9. Visual Contract и запрет default UI

Плохой визуал в IMPERIUM - не косметика, а операционный провал. Если Owner не может быстро и приятно понять состояние системы, Cockpit не выполняет работу. Поэтому визуальные задачи не могут приниматься без Visual Contract, screenshot evidence и anti-default gate.

Запрещено принимать:

- generic dashboard / bootstrap admin look;
- обычные серо-белые плоские карточки без системы;
- raw JSON или терминал как основной UX;
- green без evidence_ref;
- визуальные claims без screenshots;
- анимации без смысла, без 60 FPS/оптимизации или без reduced-motion fallback;
- игнорирование прямых визуальных требований Owner.

Требуется стремиться к:

- темной имперской кабине, metal/glass/forged feeling;
- центральному focus core / карте контуров / органам по краям;
- mission-control/NOC/SOC логике: много сигналов, но главное видно сразу;
- evidence inspector, diff, route truth, action strip;
- human compression: меньше шумных текстовых стен, больше понятной иерархии.

10. Continuity Pack как P0 функция

Build Continuity Pack - обязательная кнопка, а не later feature. Новый Logos/Servitor/Speculum не должен входить в работу вслепую. Continuity должен собираться из текущей правды, последних решений, blockers, route status, pain memory, evidence refs и next step.

Mode	Назначение	Содержимое
small	Быстрый новый чат.	START_HERE, current summary, active task, blockers, next prompt, source refs.
normal	Серьезный handoff.	current truth, last commits, active task, decisions, pain memory, routes, evidence links.
full_archive	Глубокий перенос/локальная LLM/архив.	reports, screenshots, receipts, taskpacks, PDFs, manifests, archive index.

11. Repo hygiene, archive и quarantine

NewGen не должен превращаться в свалку. Каждый артефакт должен иметь класс и место. Generated/runtime файлы не становятся canon случайно.

Class	Смысл
CANON	Настоящие законы, контракты, stable source.
CURRENT_TRUTH	Актуальные generated indexes и state, которым разрешено быть current truth.
REPORT	Evidence конкретной задачи, не всегда authority.
RUNTIME	Временный вывод, обычно не коммитится.
ARCHIVE	История для будущего анализа, не current truth.
QUARANTINE	Подозрительное, stale, duplicate, непринятое.
LOCAL_PRIVATE	Вне repo: private/local/heavy/secrets.

12. Pain Memory

Боль Owner не должна повторяться как разговор. Она должна становиться gates, prompts, validators и запретами. Pain Memory должна иметь human-readable формы и machine-readable registry.

- PAIN_VISUAL_DEFAULT_DASHBOARD -> блокирует UI task без Visual Contract и screenshots;
- PAIN_FOUNDATION_ONLY_PASS -> блокирует PASS без Owner-visible gain;
- PAIN_REPO_TRASH -> требует file classification;
- PAIN_CONTEXT_LOSS -> требует continuity pack;
- PAIN_DECORATIVE_ORGANS -> требует organ directive/veto/checklist;
- PAIN_RESOURCE_BURN_LOW_DELTA -> требует KPD plan и stop criteria.

13. Organ-Agent model

Органы не должны быть декором. Орган считается рабочим только если он влияет на разрешения, запреты, проверки, план Servitor, stop conditions или запись урока.

Орган	Обязательный смысл
Doctrinarium	Источник законов, Declaration of Form, Pain Doctrine, PASS/FAIL authority.
Mechanicus	Инструменты, validators, scripts, tool admission, route mechanics, technical checks.

Administratum	Current truth, reports, archive, continuity, ledgers, provenance, state indexes.
Officio Agentis	Поли, response contracts, permissions, language/live mode, agent discipline.
Inquisition	Anti-fake-green, scope violations, suspicious artifacts, evidence gaps.
Custodes	Защита ядра, admission to core, attack learning, approval guard.
Astronomicon	Task intake, decomposition, route planning, stage map.
Strategium	Priorities, sequencing, resource/KPD strategy, campaign direction.
Schola	Learning, training, skill gaps, lesson writeback.
Throne/Core	Будущий чистый управляющий агент, впитывающий только approved truth.

14. Обязательный путь задачи через Doctrinarium и органы

Будущая задача не должна начинаться из памяти агента. Servitor обязан читать authority source через Doctrinarium и явно сообщать, какие законы применяются. Затем он должен получать органские директивы, когда они применимы.

1. Read Doctrinarium Declaration of Form.
2. Read active Pain Doctrine.
3. Report applicable laws and gates.
4. Identify relevant organs.
5. Load organ directives/checklists/veto conditions.
6. State PASS/FAIL criteria before editing.
7. Execute bounded task.
8. Write evidence/receipts/lessons.
9. Update memory/gates/scripts only inside allowed scope.
10. Close with scoped verdict and not_proven.

15. Resource / KPD / cost awareness

Owner не должен сгорать. IMPERIUM должен показывать цену действия: время, команды, контуры, CPU/RAM/GPU/VRAM, контекст, объем артефактов, tokens where known, KPD and bottleneck. Если задача сжигает много ресурсов и дает малый visible delta, это отдельный pain event и повод к остановке/разбору.

16. Focused Local Strike method

NewGen должен учиться работать не бесконечной полировкой, а сильными локальными ударами. Focused local strike - это маленькое, точно направленное изменение, которое сразу рождает устойчивый механизм, gate, script, UI function или evidence path.

1. Выбрать один operational gap.
2. Ограничить touched paths.
3. Перед началом назвать Owner-visible gain.
4. Сформулировать PASS/FAIL.
5. Сделать минимальное рабочее изменение.
6. Доказать screenshots/validators/evidence.
7. Записать lesson/gate/script.

8. Не расширять scope без Owner.

17. PASS criteria

PASS в NewGen запрещено использовать generic-формой. Только scoped verdict.

- PASS_FOR_X_ONLY, где X точно описывает доказанную область;
- есть evidence paths и они читаемы;
- есть not_proven / not_run;
- нет scope escape;
- есть Owner-visible gain или явное инфраструктурное основание;
- есть Context Source Mix и KPD slice;
- есть чистый git closure или честный WARN/BLOCK.

18. FAIL / BLOCK criteria

- заявлено больше, чем доказано;
- green без evidence_ref;
- foundation-only подан как продукт;
- UI выглядит default при визуальной задаче;
- кнопка не пишет request/result receipt;
- нет continuity path там, где он был обязателен;
- репо засорено unclassified artifacts;
- органные ответы декоративны и не влияют на execution;
- Servitor не объяснил blocker или ресурсный burn;
- Doctrinarium/Declaration не прочитаны, хотя задача обязана была это сделать.

19. Fake Green criteria

- PASS без receipts/validators/source refs;
- UI зеленый при UNKNOWN, STALE, FOUNDATION_ONLY или NOT_WIRED;
- production/autonomy claim на базе bounded demo;
- report говорит OK, но current truth/README/index противоречат;
- screenshots отсутствуют при visual claim;
- button существует, но action receipt отсутствует;
- орган claimed active, но нет veto/checklist/prompt delta.

20. Immediate doctrine workflow

1. Принять Servitor draft как материал, не как канон.
2. Использовать этот Owner-corrected PDF как кандидата Декларации Формы.
3. Зарегистрировать в Doctrinarium human PDF + machine mirror.
4. Заставить каждый task читать Declaration through Doctrinarium.
5. Следующим отдельным task дать Servitor-Speculum red-team gap map: что мешает NewGen стать этой формой.

- 6. Из gap map сделать Declaration of Pain / Pain Doctrine.
- 7. Pain Doctrine подключить как preflight gate.

21. Ближайшие focused strikes

Приоритет	Удар	Зачем
P0	Doctrinarium Declaration registration	Чтобы Servitor читал форму из authority source, а не из loose memory.
P0	Speculum Declaration gap red-team	Получить карту разрыва между desired form и реальностью.
P0	Declaration of Pain	Превратить боль Owner в gates.
P0	Continuity Pack builder	Убрать потерю контекста и handoff pain.
P0	Visual Contract Gate	Остановить default UI и визуальные fake claims.
P0	Repo Hygiene classifier	Остановить мусор, archive/quarantine/current truth confusion.
P0	Organ directive/veto packets	Сделать органы управляющей силой, а не декором.

22. Заключение

IMPERIUM New Generation должен стать чистым ядром, которое не просто производит файлы, а организует силу. Каждый агент работает в симбиозе, каждый task оставляет заботу, каждая боль становится gate, каждый green доказывается evidence, каждый орган влияет на execution, каждый результат проходит через границы чистоты. Пока это не выполнено, система не считается хорошей. Когда Owner открывает Sanctum, видит правду, контролирует действие, собирает continuity, понимает цену и может остановить ересь, IMPERIUM начинает работать.